SVM_{Active} — Support Vector Machine Active Learning for Image Retrieval

Edward Chang
Electrical & Computer Engineering
University of California
Santa Barbara, CA 93106
echang@ece.ucsb.edu

Simon Tong
Department of Computer Science
Stanford University
Sanford, CA 94305
simon.tong@cs.stanford.edu

Abstract

Relevance feedback is often a critical component when designing image databases. With these databases it is difficult to specify queries directly and explicitly. Relevance feedback interactively determinines a user's desired output or *query concept* by asking the user whether certain proposed images are relevant or not. For a relevance feedback algorithm to be effective, it must grasp a user's query concept accurately and quickly, while also asking the user to label only a small number of images. We propose the use of a *support vector machine active learning* (SVM_{Active}) algorithm for conducting effective relevance feedback for image retrieval. To support efficient query-concept learning and image retrieval, we also present our multi-resolution image-characterization and high-dimensional indexing methods. We further show that SVM_{Active} can be effectively seeded by MEGA, another active learning algorighm that we developed, or by keyword searches. Experimental results show that our algorithm achieves significantly higher search accuracy than traditional query refinement schemes after just three to four rounds of relevance feedback.

Keywords: active learning, image retrieval, query concept, relevance feedback, support vector machines.

1 Introduction

One key design task, when constructing image databases, is the creation of an effective relevance feedback component. While it is sometimes possible to arrange images within an image database by creating a hierarchy, or by hand-labeling each image with descriptive words, these methods are time-consuming, costly, and subjective. Alternatively, requiring the end-user to specify an image query in terms of low level features (such as color and spatial relationships) is challenging to the end-user, because an image query is hard to articulate, and articulation can vary from one user to another.

Thus, we need a way for a user to implicitly inform a database of his or her desired output or *query concept*. To address this requirement, relevance feedback can be used as a query refinement scheme to derive or learn a user's query concept. To solicit feedback, the refinement scheme displays a few image instances and the user labels each image as "relevant" or "not relevant." Based on the responses, another set of images from the database is presented to the user for labeling. After a few such querying rounds, the refinement scheme returns a number of items from the database that seem to fit the needs of the user.

The construction of such a query refinement scheme (hereafter called a *query concept learner* or *learner*) can be regarded as a machine learning task. In particular, it can be seen as a case of *pool-based active learning* [19, 22]. In pool-based active learning the learner has access to a pool of unlabeled data and can request the user's label for a certain number of instances in the pool. In the image retrieval domain, the unlabeled pool would be the entire database of images. An instance would be an image, and the two possible labelings for each image would be "relevant" or "not relevant." The goal for the learner is to learn the user's *query concept* — in other words, to label each image within the database in such a manner that the learner's labeling and the user's labeling will agree.

The main issue with active learning is finding a method for choosing informative images within the pool to ask the user to label. We call such a request for the label of an image a *pool-query*. Most machine learning algorithms are *passive* in the sense that they are generally applied using a randomly selected training set. The key idea with *active* learning is that it should choose its next pool-query based upon the past answers to previous pool-queries.

In general, and for the image retrieval task in particular, such a learner must meet two critical design goals. First, the learner must learn target concepts accurately. Second, the learner must grasp a concept quickly, with only a small number of labeled instances, since most users do not wait around to provide a great deal of feedback. In this study, we propose using a *support vector machine active learner* (SVM_{Active}) to achieve these goals. SVM_{Active} combines active learning with support vector machines (SVMs). SVMs [34, 1] have met with significant success in numerous real-world learning tasks. Like most machine learning algorithms, they use a randomly selected training set, which is not very useful in the relevance feedback setting. Recently, general

purpose methods for active learning with SVMs have been independently developed by a number of researchers [2, 28, 33]. We shall use the work and theoretical motivation of [33] on active learning with SVMs to extend the use of support vector machines to the task of relevance feedback for image databases.

Intuitively, SVM_{Active} works by combining the following three ideas:

1. SVM_{Active} regards the task of learning a target concept as one of learning an SVM binary classifier. An SVM captures the query concept by separating the relevant images from irrelevant ones with a hyperplane in a projected space, usually a very high-dimensional one. The projected points on one side of the hyperplane are considered relevant to the query concept and the rest irrelevant.

2. SVM_{Active} learns the classifier **quickly** via active learning. The active part of SVM_{Active} selects the most informative instances with which to train the SVM classifier. This step ensures fast convergence to the query concept in a small number of feedback rounds.

3. Once the classifier is trained, SVM_{Active} returns the top-k most relevant images. These are the k images farthest from the hyperplane on the query concept side.

SVM_{Active} needs at least one positive and one negative example to start its learning process. We propose two seeding methods: by MEGA [4], and by keywords (Section 6). To make both concept-learning and image retrieval efficient, we employ a multi-resolution image-feature extractor (Section 5.1), and a high-dimensional indexer (Section 5.2). Through examples and empirical study, we show that combining SVM_{Active} with these other components produces a search engine particularly well suited to the query refinement task in image retrieval, significantly outperforming traditional systems.

The rest of this paper is organized into seven sections. Section 2 introduces SVMs. Section 3 then introduces the notion of a *version space* which in Section 4 provides theoretical motivation for a method of performing active learning with SVMs. Section 5 depicts our multi-resolution image characterization and high-dimensional indexer. Section 6 presents two options of finding relevant images before switching to $SVMActive$. In Section 7, we report experimental results

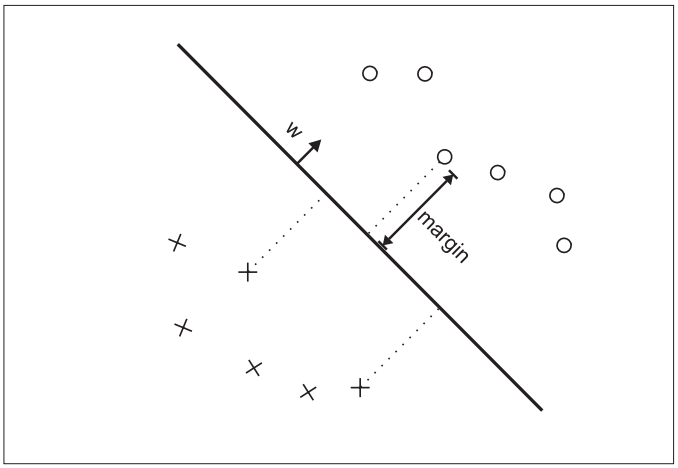


Figure 1: A simple linear Support Vector Machine

showing that our SVM active learner significantly outperforms traditional methods. Section 8 surveys related work. Finally, we offer our conclusions in Section 9.

2 Support Vector Machines

Support vector machines are a core machine learning technology. They have strong theoretical foundations and excellent empirical successes. They have been applied to tasks such as handwritten digit recognition [35], object recognition [25], and text classification [16].

We shall consider SVMs in the binary classification setting. We are given training data $\{\mathbf{x_1} \ldots \mathbf{x_n}\}$ that are vectors in some space $\mathsf{X} \subseteq \mathbb{R}^{\mathsf{d}}$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are hyperplanes that separate the training data by a maximal margin (see Fig. 1). All vectors lying on one side of the hyperplane are labeled as -1, and all vectors lying on the other side are labeled as 1. The training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow us to project the original training data in space X to a higher dimensional feature space F via a Mercer kernel operator K. In other words, we consider the set of classifiers of the form: $f(\mathbf{x}) = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{K}(\mathbf{x_i}, \mathbf{x})$. When $f(\mathbf{x}) \geq \mathbf{0}$ we classify $\mathbf{x}$ as $+1$, otherwise we classify $\mathbf{x}$ as -1.

When K satisfies Mercer's condition [1] we can write: $K(\mathbf{u}, \mathbf{v}) = \mathbf{\Phi}(\mathbf{u}) \cdot \mathbf{\Phi}(\mathbf{v})$ where $\Phi : \mathsf{X} \rightarrow \mathsf{F}$

and "$\cdot$" denotes an inner product. We can then rewrite f as:

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x}), \text{ where } \mathbf{w} = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{\Phi}(\mathbf{x_i}). \tag{1}$$

Thus, by using K we are implicitly projecting the training data into a different (often higher dimensional) feature space F. The SVM then computes the α_is that correspond to the maximal margin hyperplane in F. By choosing different kernel functions we can implicitly project the training data from X into space F. (Hyperplanes in F correspond to more complex decision boundaries in the original space X.)

Two commonly used kernels are the polynomial kernel $K(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v} + \mathbf{1})^\mathrm{p}$, which induces polynomial boundaries of degree p in the original space X, and the radial basis function kernel $K(\mathbf{u}, \mathbf{v}) = (\mathbf{e}^{-\gamma(\mathbf{u}-\mathbf{v})\cdot(\mathbf{u}-\mathbf{v})})$, which induces boundaries by placing weighted Gaussians upon key training instances. In the remainder of this paper we will assume that the modulus of the training data feature vectors are constant, i.e., for all training instances $\mathbf{x_i}$, $\|\Phi(\mathbf{x_i})\| = \lambda$ for some fixed λ. The quantity $\|\Phi(\mathbf{x_i})\|$ is always constant for radial basis function kernels, and so the assumption has no effect for this kernel. For $\|\Phi(\mathbf{x_i})\|$ to be constant with the polynomial kernels we require that $\|\mathbf{x_i}\|$ be constant. It is possible to relax this constraint on $\Phi(\mathbf{x_i})$. We shall discuss this option at the end of Section 4.

3 Version Space

Given a set of labeled training data and a Mercer kernel K, there is a set of hyperplanes that separate the data in the induced feature space F. We call this set of consistent hyperplanes or *hypotheses* the *version space* [23]. In other words, hypothesis f is in version space if for every training instance $\mathbf{x_i}$ with label y_i we have that $f(\mathbf{x_i}) > \mathbf{0}$ if $y_i = 1$ and $f(\mathbf{x_i}) < \mathbf{0}$ if $y_i = -1$. More formally:

Definition 3.1 *Our set of possible hypotheses is given as:*

$$\mathsf{H} = \left\{ \mathsf{f} \mid \mathsf{f}(\mathbf{x}) = \frac{\mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x})}{\|\mathbf{w}\|} \textit{ where } \mathbf{w} \in \mathsf{W} \right\},$$

where our parameter space W *is simply equal to* F. *The* Version space, V *is then defined as:*

$$\mathsf{V} = \{\mathsf{f} \in \mathsf{H} \mid \forall \mathsf{i} \in \{1 \ldots \mathsf{n}\} \quad \mathsf{y_i f}(\mathbf{x_i}) > \mathbf{0}\}.$$



(a) (b) (c)

Figure 2: (a) Version space duality. The surface of the hypersphere represents unit weight vectors. Each of the two hyperplanes corresponds to a labeled training instance. Each hyperplane restricts the area on the hypersphere in which consistent hypotheses can lie. Here version space is the surface segment of the hypersphere closest to the camera. (b) An SVM classifier in version space. The dark embedded sphere is the largest radius sphere whose center lies in version space and whose surface does not intersect with the hyperplanes. The center of the embedded sphere corresponds to the SVM, its radius is the margin of the SVM in F, and the training points corresponding to the hyperplanes that it touches are the support vectors. (c) Simple Margin Method.

Notice that since H *is a set of hyperplanes, there is a bijection (an exact correspondence) between unit vectors* $\mathbf{w}$ *and hypotheses* f *in* H*. Thus we will redefine* V *as:*

$$\mathsf{V} = \{\mathbf{w} \in \mathsf{W} \mid \|\mathbf{w}\| = \mathbf{1},\ \mathbf{y_i}(\mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x_i})) > \mathbf{0}, \mathbf{i} = \mathbf{1} \ldots \mathbf{n}\}.$$

Note that a version space exists only if the *training* data are linearly separable in the feature space. Thus, we require linear separability of the training data in the feature space. This restriction is much less harsh than it might at first seem. First, the feature space often has a very high dimension and so in many cases it results in the data set being linearly separable. Second, as noted by [30], it is possible to modify any kernel so that the data in the newly induced feature space is linearly separable. This is done by redefining all training instances $\mathbf{x_i}$: $K(\mathbf{x_i}, \mathbf{x_i}) \leftarrow \mathbf{K}(\mathbf{x_i}, \mathbf{x_i}) + \nu$ where ν is a positive regularization constant. The effect of this modification is to permit linear non-separability of the training data in the original feature space.

There exists a duality between the feature space F and the parameter space W [35, 13] which we shall take advantage of in the next section: points in F correspond to hyperplanes in W and *vice versa*.

Clearly, by definition, points in W correspond to hyperplanes in F. The intuition behind the converse is that observing a training instance $\mathbf{x_i}$ in feature space restricts the set of separating hyperplanes to ones that classify $\mathbf{x_i}$ correctly. In fact, we can show that the set of allowable points

$\mathbf{w}$ in W is restricted to lie on one side of a hyperplane in W. More formally, to show that points in F correspond to hyperplanes in W, suppose we are given a new training instance $\mathbf{x_i}$ with label y_i. Then any separating hyperplane must satisfy $y_i(\mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x_i})) > \mathbf{0}$. Now, instead of viewing $\mathbf{w}$ as the normal vector of a hyperplane in F, think of $y_i\Phi(\mathbf{x_i})$ as being the normal vector of a hyperplane in W. Thus $y_i(\mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x_i})) = \mathbf{w} \cdot \mathbf{y_i}\mathbf{\Phi}(\mathbf{x_i}) > \mathbf{0}$ defines a half-space in W. Furthermore $\mathbf{w} \cdot \mathbf{y_i}\mathbf{\Phi}(\mathbf{x_i}) = \mathbf{0}$ defines a hyperplane in W that acts as one of the boundaries to version space V. Notice that version space is a connected region on the surface of a hypersphere in parameter space. See Fig.2(a) for an example.

SVMs find the hyperplane that maximizes the margin in feature space F. One way to pose this is as follows:

$$\begin{aligned} \text{maximize}_{\mathbf{w} \in \mathsf{F}} \quad & \min_i\{y_i(\mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x_i}))\} \\ \text{subject to:} \quad & \|\mathbf{w}\| = \mathbf{1} \\ & y_i(\mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x_i})) > \mathbf{0} \quad \mathbf{i} = \mathbf{1} \ldots \mathbf{n}. \end{aligned}$$

By having the conditions $\|\mathbf{w}\| = \mathbf{1}$ and $y_i(\mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x_i})) > \mathbf{0}$ we cause the solution to lie in version space. Now, we can view the above problem as finding the point $\mathbf{w}$ in version space that maximizes the distance $\min_i\{\mathbf{w} \cdot \mathbf{y_i}\mathbf{\Phi}(\mathbf{x_i})\}$. From the duality between feature and parameter space, and since $\|\Phi(\mathbf{x_i})\| = \mathbf{1}$, then each $y_i\Phi(\mathbf{x_i})$ is a unit normal vector of a hyperplane in parameter space and each of these hyperplanes delimits the version space. Thus we want to find the point in version space that maximizes the minimum distance to any of the delineating hyperplanes. That is, SVMs find the center of the largest radius hypersphere whose center can be placed in version space and whose surface does not intersect with the hyperplanes corresponding to the labeled instances, as in Figure 2(b). It can be easily shown that the hyperplanes touched by the maximal radius hypersphere correspond to the support vectors and that the radius of the hypersphere is the margin of the SVM.

4 Active Learning

In pool-based active learning we have a pool of unlabeled instances. It is assumed that the instances $\mathbf{x}$ are independently and identically distributed according to some underlying distribution

$F(\mathbf{x})$, and the labels are distributed according to some conditional distribution $P(y \mid \mathbf{x})$.

Given an unlabeled pool U, an *active learner* ℓ has three components: (f, q, X). The first component is a classifier, $f : \mathsf{X} \to \{-1, 1\}$, trained on the current set of labeled data X (and possibly unlabeled instances in U too). The second component $q(X)$ is the querying function that, given a current labeled set X, decides which instance in U to query next. The active learner can return a classifier f after each pool-query (*online learning*) or after some fixed number of pool-queries.

The main difference between an active learner and a regular passive learner is the querying component q. This brings us to the issue of how to choose the next unlabeled instance in the pool to query. We use an approach that queries such instances in order to reduce the size of the version space as much as possible. We need one more definition before we can proceed:

Definition 4.1 *Area*(V) *is the surface area that the version space* V *occupies on the hypersphere* $\|\mathbf{w}\| = \mathbf{1}$.

We wish to reduce the version space as fast as possible. Intuitively, one good way of doing this is to choose a pool-query that halves the version space. More formally, we can use the following lemma to motivate which instances to use as our pool-query:

Lemma 4.2 *(Tong & Koller, 2000) Suppose we have an input space* X*, finite dimensional feature space* F *(induced via a kernel* K*), and parameter space* W*. Suppose active learner* ℓ^* *always queries instances whose corresponding hyperplanes in parameter space* W *halves the area of the current version space. Let* ℓ *be any other active learner. Denote the version spaces of* ℓ^* *and* ℓ *after* i *pool-queries as* V_i^* *and* V_i *respectively. Let* P *denote the set of all conditional distributions of* y *given* $\mathbf{x}$*. Then,*

$$\forall i \in \mathbb{N}^+ \quad \sup_{P \in \mathsf{P}} E_P[Area(\mathsf{V}_i^*)] \leq \sup_{\mathsf{P} \in \mathsf{P}} \mathsf{E}_\mathsf{P}[Area(\mathsf{V}_i)],$$

with strict inequality whenever there exists a pool-query $j \in \{1 \ldots i\}$ *by* ℓ *that does not halve version space* V_{j-1}.

This lemma says that, for any given number of pool-queries, ℓ^* minimizes the maximum

expected size of the version space, where the maximum is taken over all conditional distributions of y given $\mathbf{x}$.

Now, suppose $\mathbf{w}^* \in \mathsf{W}$ is the unit parameter vector corresponding to the SVM that we would have obtained had we known the actual labels of *all* of the data in the pool. We know that $\mathbf{w}^*$ must lie in each of the version spaces $\mathsf{V_1} \supset \mathsf{V_2} \supset \mathsf{V_3} \ldots$, where $\mathsf{V_i}$ denotes the version space after i pool-queries. Thus, by shrinking the size of the version space as much as possible with each pool-query we are reducing as fast as possible the space in which $\mathbf{w}^*$ can lie. Hence, the SVM that we learn from our limited number of pool-queries will lie close to $\mathbf{w}^*$.

This discussion provides motivation for an approach in which we query instances that split the current version space into two equal parts insofar as possible. Given an unlabeled instance $\mathbf{x}$ from the pool, it is not practical to explicitly compute the sizes of the new version spaces V^- and V^+ (i.e., the version spaces obtained when $\mathbf{x}$ is labeled as -1 and $+1$ respectively). There is a way of approximating this procedure as noted by [33]:

Simple **Method.** Recall from Section 3 that, given data $\{\mathbf{x_1} \ldots \mathbf{x_i}\}$ and labels $\{y_1 \ldots y_i\}$, the SVM unit vector $\mathbf{w_i}$ obtained from this data is the center of the largest hypersphere that can fit inside the current version space $\mathsf{V_i}$. The position of $\mathbf{w_i}$ in the version space $\mathsf{V_i}$ clearly depends on the shape of the region $\mathsf{V_i}$; however, it is often approximately in the center of the version space. Now, we can test each of the unlabeled instances $\mathbf{x}$ in the pool to see how close their corresponding hyperplanes in W come to the centrally placed $\mathbf{w_i}$. The closer a hyperplane in W is to the point $\mathbf{w_i}$, the more centrally it is placed in version space, and the more it bisects version space. Thus we can pick the unlabeled instance in the pool whose hyperplane in W comes closest to the vector $\mathbf{w_i}$. For each unlabeled instance $\mathbf{x}$, the shortest distance between its hyperplane in W and the vector $\mathbf{w_i}$ is simply the distance between the feature vector $\Phi(\mathbf{x})$ and the hyperplane $\mathbf{w_i}$ in F — which is easily computed by $|\mathbf{w_i} \cdot \Phi(\mathbf{x})|$. This results in the natural Simple rule:

- Learn an SVM on the existing labeled data and choose as the next instance to query the pool instance that comes closest to the hyperplane in F.

Figure 2(c) presents an illustration. In the stylized picture we have flattened out the surface of the unit weight vector hypersphere that appears in Figure 2(a). The white area is version

space $\mathsf{V_i}$ which is bounded by solid lines corresponding to labeled instances. The five dotted lines represent unlabeled instances in the pool. The circle represents the largest radius hypersphere that can fit in the version space. Note that the edges of the circle do not touch the solid lines — just as the dark sphere in Figure2(b) does not meet the hyperplanes on the surface of the larger hypersphere (they meet somewhere under the surface). The instance $\mathbf{b}$ is closest to the SVM $\mathbf{w_i}$ and so we will choose to query $\mathbf{b}$.

As noted by [33] there exist more sophisticated approximations that we can perform. However, these methods are significantly more computationally intensive. For the task of relevance feedback, a fast response time for determining the next image to present to the user is so critically important that these other approximations are not practical.

Our SVM_{Active} image retrieval system uses radial basis function kernels. As noted in Section 2, radial basis function kernels have the property that $\|\Phi(\mathbf{x_i})\| = \lambda$. The Simple querying method can still be used with other kernels when the training data feature vectors do not have a constant modulus, but the motivating explanation no longer holds since the SVM can no longer be viewed as the center of the largest allowable sphere. However, alternative motivations have recently been proposed by Campbell, Cristianini and Smola (2000) that do not require a constraint on the modulus.

For the image retrieval domain, we also have a need for performing multiple pool-queries at the same time. It is not practical to present one image at a time for the user to label, because he or she is likely to lose patience after a few rounds. To prevent this from happening we present the user with multiple images (say, twenty) at each round of pool-querying. Thus, for each round, the active learner has to choose not just one image to be labeled but twenty. Theoretically it would be possible to consider the size of the resulting version spaces for each possible labeling of each possible set of twenty pool-queries, but clearly this would be impractical. Thus instead, for matters of computational efficiency, our SVM_{Active} system takes the simple approach of choosing the pool-queries to be the twenty images closest to its separating hyperplane.

It has been noted [33] that the Simple querying algorithm used by SVM_{Active} can be unstable during the first round of querying. To address this issue, SVM_{Active} always randomly chooses twenty images for the first relevance feedback round. Then it uses the Simple active querying

Filter Name	Resolution	Representation
Masks	Coarse	Appearance of culture colors
Spread	Coarse	Spatial concentration of a color
Elongation	Coarse	Shape of a color
Histograms	Medium	Distribution of colors
Average	Medium	Similarity comparison within the same culture color
Variance	Fine	Similarity comparison within the same culture color

Table 1: Multi-resolution Color Features.

method on the second and subsequent rounds.

SVM_{Active} Algorithm Summary

To summarize, our SVM_{Active} system performs the following for each round of relevance feedback:

- Learn an SVM on the current labeled data
- If this is the first feedback round, ask the user to label twenty randomly selected images. Otherwise, ask the user to label the twenty pool images closest to the SVM boundary.

After the relevance feedback rounds have been performed, SVM_{Active} retrieves the top-k most relevant images:

- Learn a final SVM on the labeled data.
- The final SVM boundary separates "relevant" images from irrelevant ones. Display the k "relevant" images that are farthest from the SVM boundary.

5 Indexing Image Features

We describe how our system characterizes images and how we index image features. Multi-resolution image characterization makes learning effective and efficient. Our high-dimensional indexing scheme makes image retrieval efficient.

5.1 Image Characterization

We believe that image characterization should follow human perception [12]. In particular, our perception works in a multi-resolution fashion. For some visual tasks, human eyes may select coarse filters to obtain coarse image features; for others, they select finer features. Similarly, for some image applications (e.g., for detecting image replicas), employing coarse features is sufficient; for other applications (e.g., for classifying images), employing finer features may be essential. An image search engine thus must have the flexibility to model subjective perceptions and to satisfy a variety of search tasks.

Our image retrieval system employs a multi-resolution image representation scheme [5]. In this scheme, we characterize images by two main features: color and texture. We consider shape as an attribute of these main features.

5.1.1 Color

Although the wavelength of visible light ranges from 400 nanometers to 700 nanometers, research [12] shows that the colors that can be named by all cultures are generally limited to eleven. In addition to *black* and *white*, the discernible colors are *red*, *yellow*, *green*, *blue*, *brown*, *purple*, *pink*, *orange* and *gray*.

We first divide color into 12 color bins including 11 bins for culture colors and one bin for outliers [14]. At the coarsest resolution, we characterize color using a color mask of 12 bits. To record color information at finer resolutions, we record eight additional features for each color. These eight features are color histograms, color means (in H, S and V channels), color variances (in H, S and V channel), and two shape characteristics: elongation and spreadness. Color elongation characterizes the shape of a color, and spreadness characterizes how that color scatters within the image [17]. Table 1 summarizes color features in coarse, medium and fine resolutions.

5.1.2 Texture

Texture is an important cue for image analysis. Studies [21, 31, 32, 20] have shown that characterizing texture features in terms of structuredness, orientation, and scale (coarseness) fits well

with models of human perception. A wide variety of texture analysis methods have been proposed in the past. We choose a discrete wavelet transformation (DWT) using quadrature mirror filters [31] because of its computational efficiency.

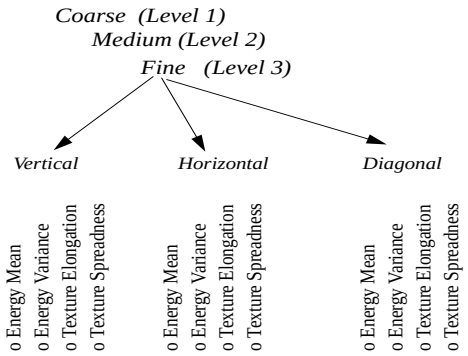


Figure 3: Multi-resolution Texture Features.

Each wavelet decomposition on a 2-D image yields four subimages: a $\frac{1}{2} \times \frac{1}{2}$ scaled-down image of the input image and its wavelets in three orientations: horizontal, vertical and diagonal. Decomposing the scaled-down image further, we obtain the tree-structured or wavelet packet decomposition. The wavelet image decomposition provides a representation that is easy to interpret. Every subimage contains information of a specific scale and orientation and also retains spatial information. We obtain nine texture combinations from subimages of three scales and three orientations. Since each subimage retains the spatial information of texture, we also compute elongation and spreadness for each texture channel. Figure 3 summarizes texture features.

Now, given an image, we can extract the above color and texture information to produce a 144 dimensional vector of numbers. We use this vector to represent the image. Thus, the space X for our SVMs is a 144 dimensional space, and each image in our database corresponds to a point in this space.

5.2 High-dimensional Indexer

An indexer is essential to make both query-concept learning and image retrieval efficient and scalable. Our indexing method is a statistical approach that works in two steps. It first performs non-supervised clustering using Tree-Structured Vector Quantization (TSVQ) [10] to group similar

objects. To maximize IO efficiency, each cluster is stored in a sequential file. For details, please consult [11].

The indexing structure is helpful for both selecting samples and retrieving images given the class boundary.

- Informative example. Each image cluster is represented by its centroid. Initially we use just the centroids as unlabeled images in our pool. Since the number of cluster-centroids is substantially less than the number of data objects[1], they can be cached in main memory to avoid IOs. When SVM_{Active} considers images from the pool, it is simply accessing unlabeled centroids in main memory. The time required for selecting images can thus be drastically reduced. After using just the centroids in the pool, SVM_{Active} then adds the images in the clusters close to the current decision boundary.

- Image retrieval. Given an SVM hyperplane, our search engine tests the centroids that fall on the relevant side of the boundary. Each cluster is ranked by its centroid's distance to the SVM hyperplane: the farther away a cluster is from the hyperplane on the relevant side, the higher the priority. The search engine then reads the "farthest" cluster into memory and performs a sequential scan on the cluster to find the top-k nearest neighbors. If higher recall is desired, we read the next farthest cluster.

6 Seeding

SVM_{Active} requires at least one relevant and one irrelevant example to start its learning. Though irrelevant examples abound, relevant examples can be difficult to find. In particular, if the number of objects relevant to the query concept is substantially less than that of the irrelevant ones, we need an effective strategy to find relevant starting images.

We call the process of finding initial relevant images *seeding*, and we present three seeding options: *seeding by* MEGA, *seeding by keywords*, and *seeding by images*. Although SVM_{Active} can be seeded by any of these methods, we believe that the first two options are more effective. The

[1] The ratio of the number of clusters over the total number of objects is data-set dependent and is adjustable. A typical ratio is less than 0.5%

seeding-by-images paradigm, which relies upon users to start a search with "good" images, may not be realistic. After all, if users can always find good images to seed a search, why would they need an image search engine? In the remainder of this section, we first present our seeding-by-MEGA method, which uses another learning algorithm MEGA [4] to find relevant images. We then discuss a seeding-by-keywords alternative.

6.1 Seeding by MEGA

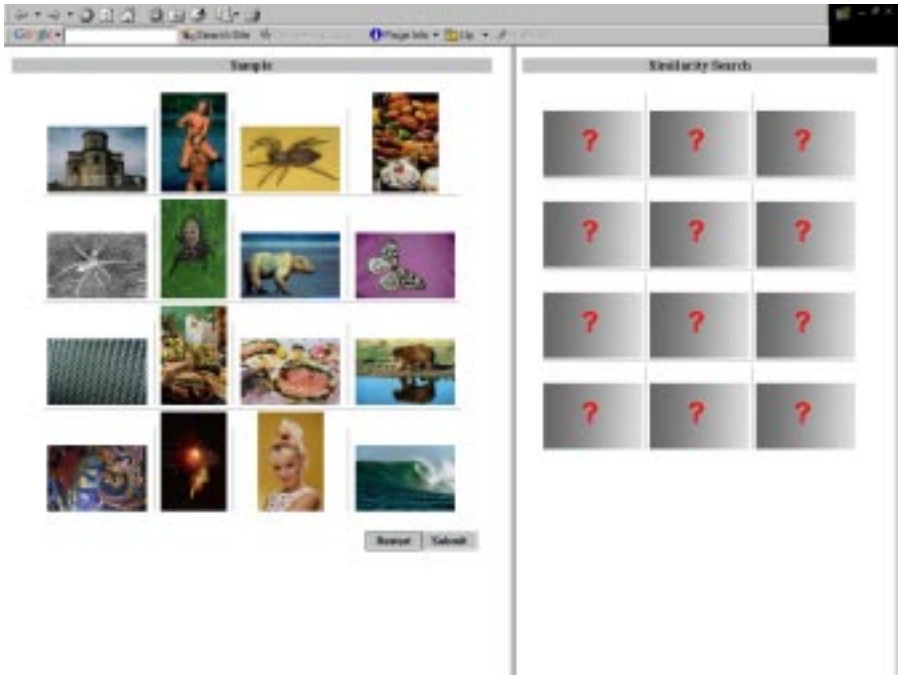
We propose using MEGA to seed SVM_{Active}. MEGA uses a boolean formula to describe query concepts. For the task of searching for an initial relevant image, MEGA uses a k-DNF formula to bound the set of permitted formulae. Thus, any image with features that satisfy the k-DNF is regarded as potentially relevant. MEGA attempts to choose images that either will be relevant, or will make the k-DNF as specific as possible. The detailed MEGA algorithm is documented in [4].

Seeding by MEGA Example

We present an example produced using a Corel image dataset to show the learning steps of SVM_{Active} seeded by MEGA for grasping a query concept "tigers." Our prototype works directly with the low-level image features described in Section 5.1. This interactive query session involves four screens that are illustrated in four figures.

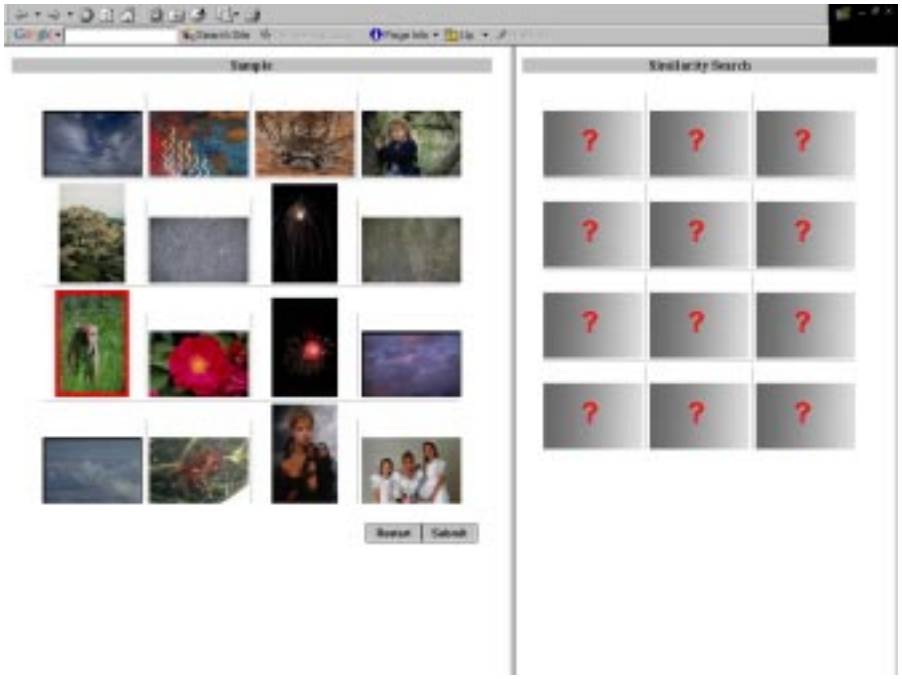
Screen 1. Initial Screen. The system presents the initial screen to the user as depicted in Figure 4(a). The screen is split into two frames vertically. On the left-hand side of the screen is the learner frame; on the right-hand side is the retrieval frame. In the learner frame, the system displays informative images so as to learn what the user wants via an active learning process. The retrieval frame displays images that the system believes are most relevant, i.e., the images that the user wants. In this first screen, the system displays a random selection of images from the database. Notice that there is no tiger image present, so we press the submit button without highlighting any images.

Screen 2. Since SVM_{Active} did not receive a relevant image in the initial round of random images, it uses MEGA to search for a single relevant seed image. Thus, MEGA now decides which images should be displayed for the user to label next. It chooses images that are rather different from



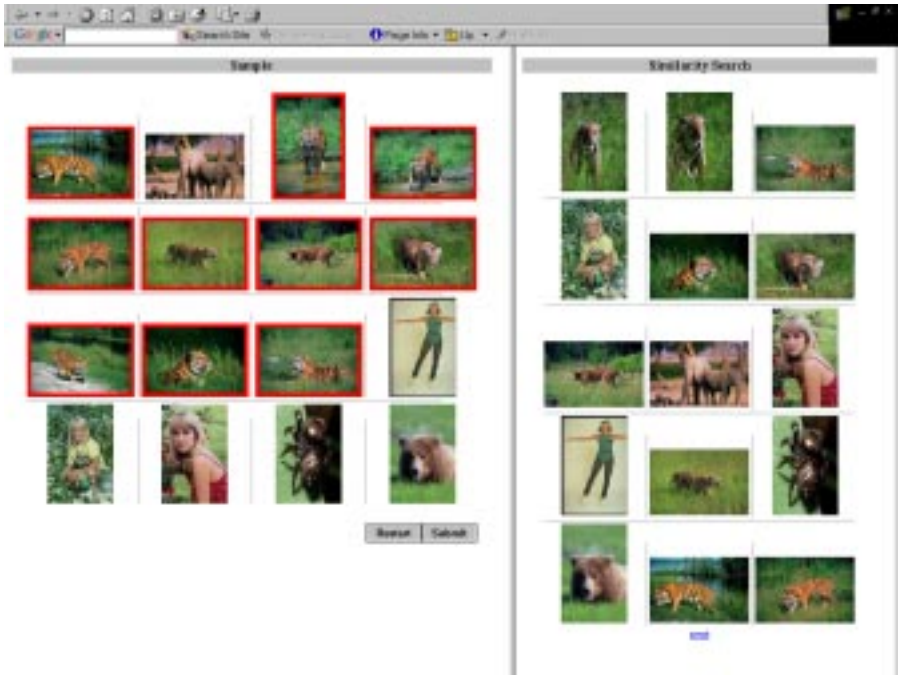
Learner Screen Retrieval Screen Screen

(a) Screen #1

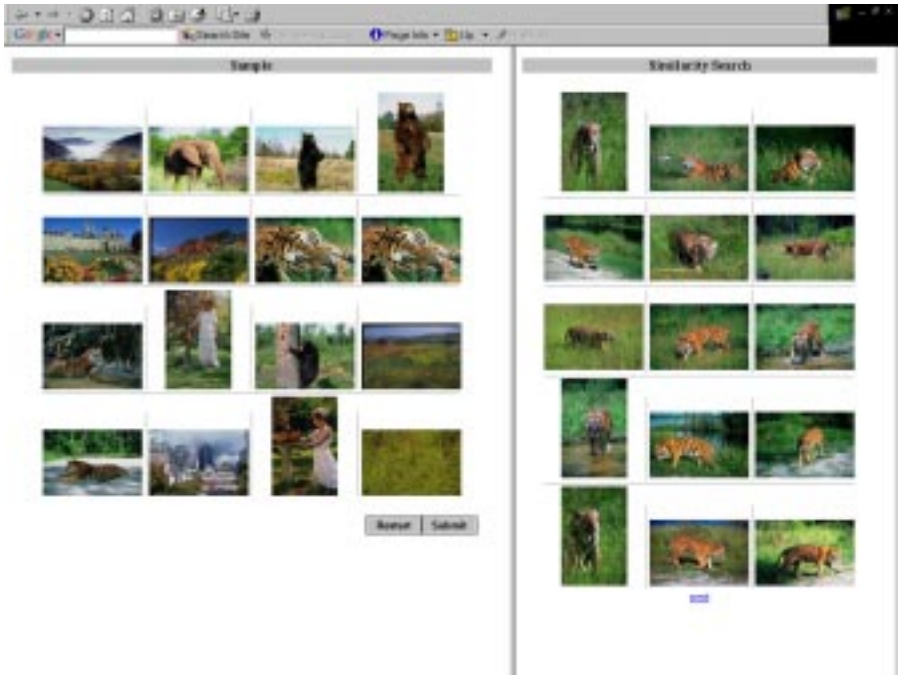


(b) Screen #2

Figure 4: Tiger Query Screens #1 and #2.



(a) Screen #3



(b) Screen #4

Figure 5: Tiger Query Screens #3 and #4.

the initial random selection. Notice that it presents a flower, clouds and tiger images. We are interested in tigers, so we highlight the tiger image as relevant, and the rest of the unmarked images are considered irrelevant. We indicate our selection by clicking on the submit button in the learner screen. This action brings up the next screen.

Screen 3. Sampling and relevance feedback continues. Now that MEGA has successfully found a relevant image, SVM_{Active} then learns an SVM using the one relevant image and 31 irrelevant images. SVM_{Active} then displays Figure 5(a). The right-hand frame displays images that the system expects will match the user's query concept at this time. As the figure indicates, with nine out of the top fifteen images already showing tigers, the system is beginning to learn which types of images we are seeking. In the left-hand frame, the system displays the images that it expects would be most informative for the user to label. The user can ask the system to further refine the target concept by selecting relevant images in this learner frame. After the user clicks on the submit button in the learner frame, the fourth screen is displayed.

Screen 4. Sampling and relevance feedback ends. Figure 5(b) shows that all returned images in the similarity search frame fit the query concept (tiger).

This example shows that by using SVM_{Active} seeded with MEGA, a query concept can be learned in very few user iterations from no initial starting knowledge. The learner is able to match high-level concepts to low-level features directly through an active learning process.

Although much work is needed to improve our learning algorithms, this prototype demonstrates a potential to facilitate information retrieval in three respects:

- *Eliminating the seeding requirement.* A user is not required to initiate a query with "good" seed images. He or she does not need to provide an example image to start the search. Even if all initial examples presented are negative, the negative-labeled instances can be used to shrink the pool of the next sampling round substantially. Therefore, the probability that a relevant example will be presented in the next iteration will be substantially higher.
- *Supporting flexible concept formulation.* One can search for a general animal concept or a specific kind of animal. In the above example, if we were to mark all wild animal images as relevant, the final result would show other wild animals as well as tigers. Also note in the third screen that the system presents tigers of different kinds and on different backgrounds, to ask for feedback.

The system can intelligently explore more possibilities to refine the query concept. (The readers are encouraged to experiment with our online demo [3].)

- *Accomplishing the above tasks quickly and accurately.* Our multi-resolution image-feature extractor and high-dimensional indexer (presented in Section 5) make both query-concept learning and image retrieval efficient and effective.

6.2 Seeding by Keyword

A text-based image retrieval engine finds images based on keywords. The keywords entered by users may have multiple senses and hence may not precisely characterize the users' query concept. There are a number of systems that perform image retrieval by keyword. For example, Figure 6 shows the search results with the keyword "baby" using the Google image search engine available at images.google.com. As observed, the returned images range from baby photos to baby furniture, statues, and cartoons.

Although keyword searches may not achieve high precision, a low-accuracy keyword search can be used instead of MEGA to seed SVM_{Active} with a single relevant image.

7 Experiments

For empirical evaluation of our learning methods, we used three real-world image datasets: a four-category, a ten-category, and a fifteen-category image dataset, each category consisting of 100 to 150 images. These image datasets were collected from Corel Image CDs and the Internet.

- *Four-category* set. The 602 images in this dataset belong to four categories — *architecture*, *flowers*, *landscape*, and *people*.
- *Ten-category* set. The 1277 images in this dataset belong to ten categories — *architecture*, *bears*, *clouds*, *flowers*, *landscape*, *people*, *objectionable images*, *tigers*, *tools*, and *waves*. In this set, a few categories were added to increase learning difficulty. The tiger category contains images of tigers with landscape and water backgrounds to complicate landscape category. The objectionable (pronographic) images can be confused with people wearing little clothing (beach wear). Clouds and waves have substantial color similarity.

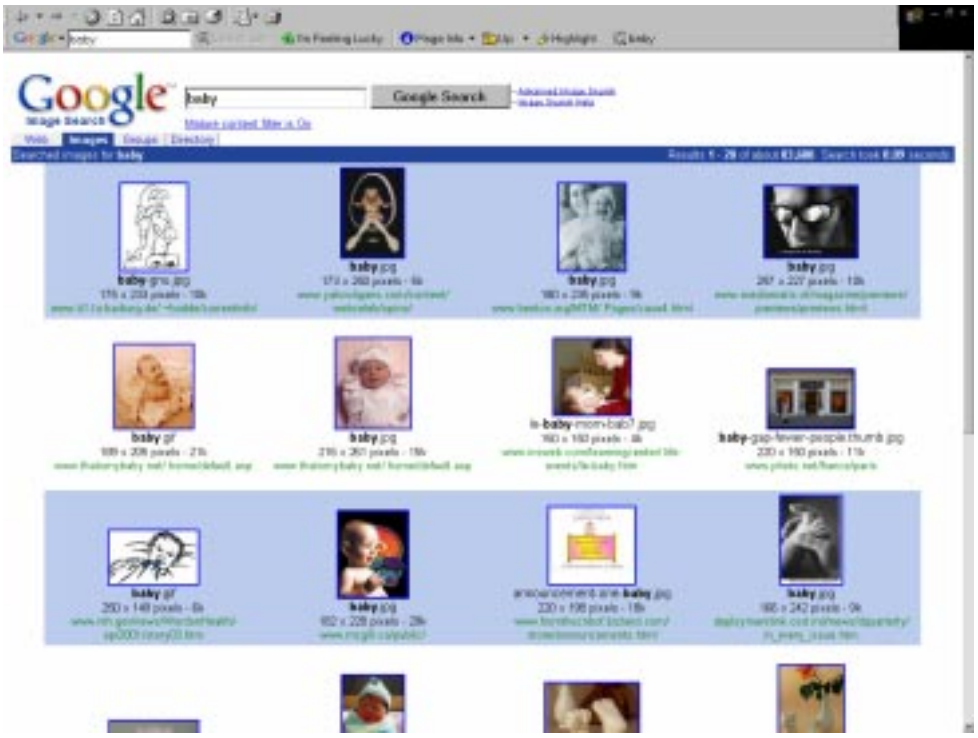


Figure 6: Google Image Search Results with Keyword "baby".

- *Fifteen-category* set. In addition to the ten categories in the above dataset, the total of 1920 images in this dataset includes *elephants*, *fabrics*, *fireworks*, *food*, and *texture*. We added elephants with landscape and water backgrounds to increase learning difficulty in distinguishing landscape, tigers and elephants. We added colorful fabrics and food to interfere with flowers. Various texture images (e.g., skin, brick, grass, water, etc.) were added to raise learning difficulty for all categories.

To obtain an objective measure of performance, we assumed that a query concept was an image category. The SVM_{Active} learner has no prior knowledge about image categories[2]. It treats each image as a 144-dimension vector described in Section 5.1. The goal of SVM_{Active} is to learn a given concept through a relevance feedback process. In this process, at each feedback round SVM_{Active} selects twenty images to ask the user to label as "relevant" or "not relevant" with respect to the query concept. It then uses the labeled instances to successively refine the concept boundary.

[2]Unlike some recently developed systems [36] that contain a semantical layer between image features and queries to assist query refinement, our system does not have an explicit semantical layer. We argue that having a hard-coded semantical layer can make a retrieval system restrictive. Rather, dynamically learning the semantics of a query concept is more flexible and hence makes the system more useful.

After finishing the relevance feedback rounds, SVM_{Active} then retrieves the top-k most relevant images from the dataset, based on the final concept it has learned. Accuracy is then computed by looking at the fraction of the k returned result that belongs to the target image category. We note that this computation is equivalent to computing the precision on the top-k images. This measure of performance appears to be the most appropriate for the image retrieval task — particularly since, in most cases, not all of the relevant images can be displayed to the user on one screen. As in the case of web searching, we typically wish the first screen of returned images to contain a high proportion of relevant images. We are less concerned that not every single instance that satisfies the query concept is displayed. As with all SVM algorithms, SVM_{Active} requires at least one relevant and one irrelevant image to function. In practice a single relevant image could be seeded into SVM_{Active} by a keyword search or by MEGA, as we discussed in Section 6. In either case our experiments assume that we start off with one randomly selected relevant image and one randomly selected irrelevant image.

7.1 SVM_{Active} Experiments

Figures 7(a-c) show the average top-k accuracy for the three different sizes of data sets. We considered the performance of SVM_{Active} after each round of relevance feedback. The graphs indicate that performance clearly increases after each round. Also, the SVM_{Active} algorithm's performance degrades gracefully when the size and complexity of the database is increased – for example, after four rounds of relevance feedback, it achieves an average of 100%, 95%, and 88% accuracy on the top-20 results for the three different sizes of data sets, respectively. It is also interesting to note that SVM_{Active} is not only good at retrieving just the top few images with high precision, but it also manages to sustain fairly high accuracy even when asked to return larger numbers of images. For example, after five rounds of querying it attains 99%, 84% and 76% accuracy on the top-70 results for the three different sizes of data sets respectively. SVM_{Active} uses the active querying method outlined in Section 4.

We examined the effect that the active querying method had on performance. Figures 8(a) and 8(b) compare the active querying method with the regular passive method of sampling. The passive method chooses images randomly from the pool to be labeled. This method is typically used with SVMs since it creates a randomly selected data set. It is clear that the use of active

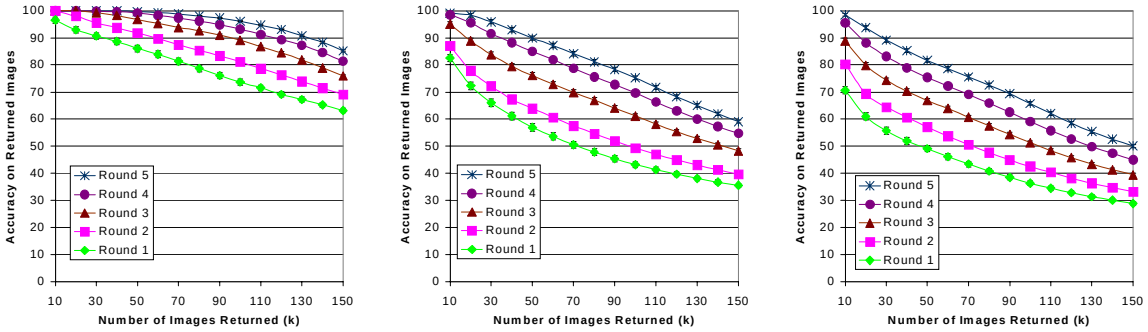
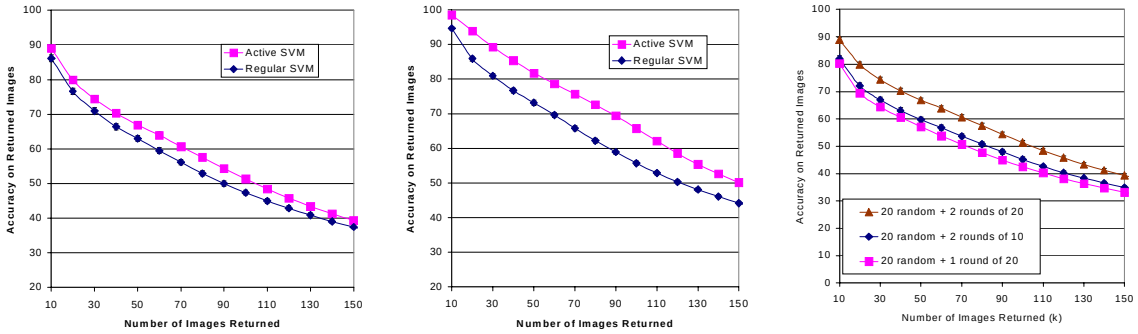


Figure 7: (a) Average top-k accuracy over the four-category dataset. (b) Average top-k accuracy over the ten-category dataset. (c) Average top-k accuracy over the fifteen-category dataset. Standard error bars are smaller than the curves' symbol size. Legend order reflects order of curves.

learning is beneficial in the image retrieval domain. We gain a significant increase in performance by using the active method. SVM_{Active} displays 20 images per pool-querying round. There is a tradeoff between the number of images to be displayed in one round, and the number of querying rounds. The fewer images displayed per round, the lower the performance. However, with fewer images per round we may be able to conduct more rounds of querying and thus increase our performance. Figure 8(c) considers the effect of displaying only ten images per round. In this experiment our first round consisted of displaying twenty random images and then, on the second and subsequent rounds of querying, active learning with 10 or 20 images is invoked. We notice that there is indeed a benefit to asking (20 random + two rounds of 10 images) over asking (20 random + one round of 20 images). This is unsurprising since the active learner has more control and freedom to adapt when asking for two rounds of 10 images rather than one round of 20. What is interesting is that asking (20 random + two rounds of 20 images) is far, far better than asking (20 random + two rounds of 10 images). The increase in the cost to users of asking 20 images per round is generally negligible since users can pick out relevant images easily. Furthermore, there is virtually no additional computational cost in calculating with the 20 images to query instead of the 10 images. Thus, for this particular task, we think it worthwhile to display around 20 images per screen and limit the number of querying rounds, rather than display fewer images per screen and require many more querying rounds.

We also investigated how performance altered when various aspects of the algorithm were



(a) (b) (c)

Figure 8: (a) Active and regular passive learning on the fifteen-category dataset after three rounds of querying. (b) Active and regular passive learning on the fifteen-category dataset after five rounds of querying. (c) Comparison between asking ten images per pool-query round and twenty images per pool-querying round on the fifteen-category dataset. Standard error bars are smaller than the curves' symbol size. Legend order reflects order of curves.

Texture features	Top-50 Accuracy
None	80.6 ± 2.3
Fine	85.9 ± 1.7
Medium	84.7 ± 1.6
Coarse	85.8 ± 1.3
All	86.3 ± 1.8

Table 2: Average top-50 accuracy over the four-category data set using a regular SVM trained on 30 images. Texture spatial features were omitted.

changed. Table 2 shows that all three of the texture resolutions are important. Also, the performance of the SVM appears to be greatest when all of the texture resolutions are included (although in this case the difference is not statistically significant). Table 3 indicates how other SVM kernel functions perform on the image retrieval task compared to the radial basis function kernel. It appears that the radial basis function kernel is the most suitable for this feature space. One other important aspect of any relevance feedback algorithm is the wall clock time that it takes to generate the next pool-queries. Relevance feedback is an interactive task, and if the algorithm takes too long then the user is likely to lose patience and be less satisfied with the experience.

	Top-50	Top-100	Top-150
Degree 2 Polynomial	95.9 ± 0.4	86.1 ± 0.5	72.8 ± 0.4
Degree 4 Polynomial	92.7 ± 0.6	82.8 ± 0.6	69.0 ± 0.5
Radial Basis	$\mathbf{96.8 \pm 0.3}$	$\mathbf{89.1 \pm 0.4}$	$\mathbf{76.0 \pm 0.4}$

Table 3: Accuracy on four-category data set after three querying rounds using various kernels. Bold type indicates statistically significant results.

Dataset	Dataset Size	round of 20 queries (secs)	Computing final SVM	Retrieving top 150 images
4 Cat	602	0.34 ± 0.00	0.5 ± 0.01	0.43 ± 0.02
10 Cat	1277	0.71 ± 0.01	1.03 ± 0.03	0.93 ± 0.03
15 Cat	1920	1.09 ± 0.02	1.74 ± 0.05	1.37 ± 0.04

Table 4: Average run times in seconds

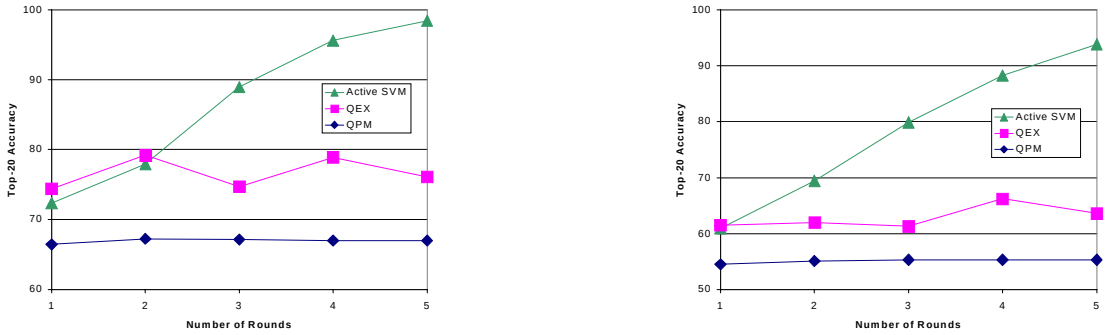


Figure 9: (a) Average top-k accuracy over the ten-category dataset. (b) Average top-k accuracy over the fifteen-category dataset.

Table 4 shows that SVM_{Active} averages about a second on a Sun Workstation to determine the 20 most informative images for the users to label. Retrieval of the 150 most relevant images takes a similar amount of time, and computing the final SVM model never exceeds two seconds.

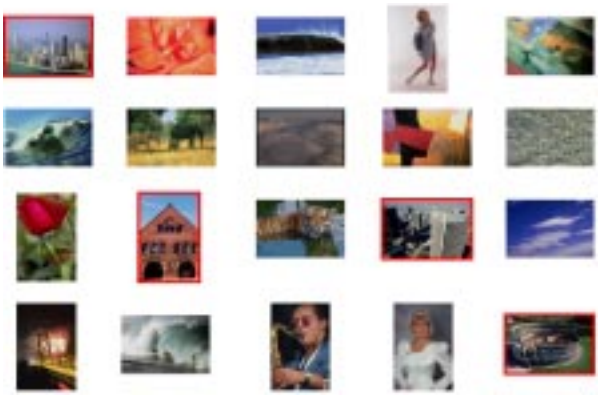
7.2 Scheme Comparison

We also compared SVM_{Active} with two traditional query refinement methods: *query point movement* (QPM) and *query expansion* (QEX). In this experiment, each scheme returned the 20 most relevant images after up to five rounds of relevance feedback. To ensure that the comparison to SVM_{Active} was fair, we seeded both schemes with one randomly selected relevant image to generate the first round of images. On the ten-category image dataset, Figure 9(a) shows that SVM_{Active} achieves nearly 90% accuracy on the top-20 results after three rounds of relevance feedback, whereas the accuracies of both QPM and QEX never reach 80%. On the fifteen-image category dataset, Figure 9(b) shows that SVM_{Active} outperforms the others by even wider margins. SVM_{Active} reaches 80% top-20 accuracy after three rounds and 94% after five rounds, whereas QPM and QEX cannot achieve 65% accuracy.

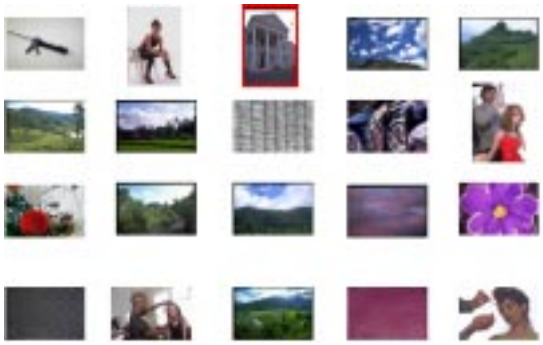
These results hardly surprise us. Traditional information retrieval schemes require a large



Initializing



Feedback Round 1



Feedback Round 2



Feedback Round 3

Figure 10: Searching for architecture images. SVM_{Active} Feedback phase.

number of image instances to achieve any substantial refinement. By just refining around current relevant instances, both QPM and QEX tend to be fairly localized in their exploration of the image space and hence rather slow in exploring the entire space. In contrast, during the relevance feedback phase SVM_{Active} takes both the relevant and irrelevant images into account when choosing the next pool-queries. Furthermore, it chooses to ask the user to label images that it regards as most **informative** for learning the query concept, rather than those that have the most likelihood of being relevant. Thus it tends to explore the feature space more aggressively.

Figures 10 and 11 show an experimental run of the SVM_{Active} system. For this run, we are interested in obtaining architecture images. In Figure 10 we initialize the search by giving SVM_{Active} one relevant and one irrelevant image. We then have three feedback rounds. The images that SVM_{Active} asks us to label in these three feedback rounds are images that SVM_{Active} will judge to be most informative. For example, we see that it asks us to label a number of landscape images and other images with a blue or gray background with something in the foreground. The feedback



First Screen of Results



Second Screen of Results



Third Screen of Results



Fourth Screen of Results



Fifth Screen of Results



Sixth Screen of Results

Figure 11: Searching for architecture images. SVM_{Active} Retrieval phase.

rounds allow SVM_{Active} to narrow down the range of images that we like. When it comes to the retrieval phase (Figure 10) SVM_{Active} returns, with high precision, a large variety of different architecture images, ranging from old buildings to modern cityscapes.

8 Related Work

There have been several studies of active learning for classification in the machine learning community. The *query by committee* algorithm [29, 9] uses a distribution over all possible classifiers and attempts to greedily reduce the entropy of this distribution. This general purpose algorithm has been applied in a number of domains[3] using classifiers (such as Naive Bayes classifiers [7, 22]) for which specifying and sampling classifiers from a distribution is natural. Probabilistic models such as the Naive Bayes classifier provide interpretable models and principled ways to incorporate prior knowledge and data with missing values. However, they typically do not perform as well as discriminative methods such as SVMs [16, 8].

Lewis and Gale (1994) focused on the text classification task and introduced uncertainty sampling using logistic regression and also decision trees [18]. SVM_{Active}'s querying method is essentially the same as their uncertainty sampling method (choose the instance that our current classifier is most uncertain about).

Relevance feedback techniques proposed by the information retrieval and database communities also perform non-random sampling. The study of [27] puts these query refinement approaches into three categories: *query reweighting*, *query point movement* and *query expansion*.

- *Query reweighting* and *query point movement* [15, 24, 26]. Both query reweighting and query point movement use nearest-neighbor sampling: They return top ranked objects to be marked by the user and refine the query based on the feedback. These methods tend to require a large number of image instances to achieve any substantial refinement since they do not tend to explore the image space aggressively.
- *Query expansion* [27, 37]. The *query expansion* approach can be regarded as a multiple-instances sampling approach. The samples of the next round are selected from the neighborhood (not

[3] Although, to our knowledge, not to the image retrieval domain.

necessarily the nearest ones) of the positive-labeled instances of the previous round. The study of [27] shows that query expansion achieves only a slim margin of improvement (about 10% in precision/recall) over query point movement.

For image retrieval, the PicHunter system [6] uses Bayesian prediction to infer the goal image, based upon the users' actions. The system shows that employing active learning can drastically cut down the number of iterations (up to 80% in some experiments). However the authors also point out that their scheme is computationally intensive, since it needs to recompute the conditional probability for all unlabeled samples after each round of user feedback and hence may not scale well with dataset size.

9 Conclusions and Future Work

We have demonstrated that active learning with support vector machines can provide a powerful tool for searching image databases, outperforming a number of traditional query refinement schemes. SVM_{Active} not only achieves consistently high accuracy on a wide variety of desired returned results, but also does it quickly and maintains high precision when asked to deliver large quantities of images. Also, unlike recent systems such as SIMPLIcity [36], it does not require an explicit semantical layer to perform well. Our system takes advantage of the intuition that there can be considerable differences between the set of images that we are already confident a user wishes to see, and the set of images that would most informative for the user to label. By decoupling the notions of feedback and retrieval, and by using a powerful classifier with active learning, we have demonstrated that SVM_{Active} can provide considerable gains over other systems.

We have also presented our image characterization and feature indexing methods, which make concept-learning and image retrieval effective and efficient. We have built a prototype, which uses MEGA to seed SVM_{Active} by finding the first image(s) relevant to the query concept, and then switches to use SVM_{Active} for the subsequent rounds of feedback. Our online demo [3] is available at: www.mmdb.ece.ucsb.edu/~demo/corelacm/.

10 Acknowledgements

Simon Tong was supported by DARPA's *Information Assurance* program under subcontract to SRI International, and by ONR MURI N00014-00-1-0637.

References

[1] C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167, 1998.

[2] C. Campbell, N. Cristianini, and A. Smola. Query learning with large margin classifiers. In *Proceedings of the Seventeenth International Conference on Machine Learning*, 2000.

[3] E. Chang, K.-T. Cheng, W.-C. Lai, C.-T. Wu, C.-W. Chang, and Y.-L. Wu. PBIR — a system that learns subjective image query concepts. *Proceedings of ACM Multimedia, http://www.mmdb.ece.ucsb.edu/~demo/corelacm/*, pages 611–614, October 2001.

[4] E. Chang and B. Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/∼echang/mega.pdf*, November 2000.

[5] E. Chang, B. Li, and C. Li. Towards perception-based image retrieval. *IEEE Content-Based Access of Image and Video Libraries*, pages 101–105, June 2000.

[6] I. J. Cox, M. L. Miller, T. P. Minka, T. V. Papathomas, and P. N. Yianilos. The bayesian image retrieval system, pichunter: Theory, implementation and psychological experiments. *IEEE Transaction on Image Processing (to appear)*, 2000.

[7] I. Dagan and S. Engelson. Committee-based sampling for training probabilistic classifiers. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 150–157. Morgan Kaufmann, 1995.

[8] S. Dumais, J. Platt, D. Heckerman, and M. Sahami. Inductive learning algorithms and representations for text categorization. In *Proceedings of the Seventh International Conference on Information and Knowledge Management*. ACM Press, 1998.

[9] Y. Freund, H. Seung, E. Shamir, and N. Tishby. Selective sampling using the Query by Committee algorithm. *Machine Learning*, 28:133–168, 1997.

[10] A. Gersho and R. Gray. *Vector Quantization and Signal Compression*. Kluwer Academic, 1991.

[11] K. Goh and E. Chang. Indexing multimedia data in high-dimensional and dynamic weighted feature spaces. *The 6th Visual Database Conference*, May 2002.

[12] E. B. Goldstein. *Sensation and Perception (5th Edition)*. Brooks/Cole, 1999.

[13] R. Herbrich, T. Graepel, and C. Campbell. Bayes point machines: Estimating the bayes point in kernel space. In *International Joint Conference on Artificial Intelligence Workshop on Support Vector Machines*, pages 23–27, 1999.

[14] K. A. Hua, K. Vu, and J.-H. Oh. Sammatch: A flexible and efficient sampling-based image retrieval technique for image databases. *Proceedings of ACM Multimedia*, November 1999.

[15] Y. Ishikawa, R. Subramanya, and C. Faloutsos. Mindreader: Querying databases through multiple examples. *VLDB*, 1998.

[16] T. Joachims. Text categorization with support vector machines. In *Proceedings of the European Conference on Machine Learning*. Springer-Verlag, 1998.

[17] J.-G. Leu. Computing a shape's moments from its boundary. *Pattern Recognition*, pages Vol.24, No.10,pp.949–957, 1991.

[18] D. Lewis and J. Catlett. Heterogeneous uncertainty sampling for supervised learning. In *Proceedings of the Eleventh International Conference on Machine Learning*, pages 148–156. Morgan Kaufmann, 1994.

[19] D. Lewis and W. Gale. A sequential algorithm for training text classifiers. In *Proceedings of the Seventeenth Annual International ACM-SIGIR Conference on Research and Development in Information Retrieval*, pages 3–12. Springer-Verlag, 1994.

[20] W. Y. Ma and H. Zhang. Benchmarking of image features for content-based retrieval. *Proceedings of Asilomar Conference on Signal, Systems & Computers*, 1998.

[21] B. Manjunath, P. Wu, S. Newsam, and H. Shin. A texture descriptor for browsing and similarity retrieval. *Signal Processing Image Communication*, 2001.

[22] A. McCallum and K. Nigam. Employing EM in pool-based active learning for text classification. In *Proceedings of the Fifteenth International Conference on Machine Learning*. Morgan Kaufmann, 1998.

[23] T. Mitchell. Generalization as search. *Artificial Intelligence*, 28:203–226, 1982.

[24] M. Ortega, Y. Rui, K. Chakrabarti, A. Warshavsky, S. Mehrotra, and T. S. Huang. Supporting ranked boolean similarity queries in mars. *IEEE Transaction on Knowledge and Data Engineering*, 10(6):905–925, December 1999.

[25] C. Papageorgiou, M. Oren, and T. Poggio. A general framework for object detection. In *Proceedings of the International Conference on Computer Vision*, 1998.

[26] K. Porkaew, K. Chakrabarti, and S. Mehrotra. Query refinement for multimedia similarity retrieval in mars. *Proceedings of ACM Multimedia*, November 1999.

[27] K. Porkaew, S. Mehrota, and M. Ortega. Query reformulation for content based multimedia retrieval in mars. *ICMCS*, pages 747–751, 1999.

[28] G. Schohn and D. Cohn. Less is more: Active learning with support vector machines. In *Proceedings of the Seventeenth International Conference on Machine Learning*, 2000.

[29] H. Seung, M. Opper, and H. Sompolinsky. Query by committee. In *Proceedings of the Fifth Workshop on Computational Learning Theory*, pages 287–294. Morgan Kaufmann, 1992.

[30] J. Shawe-Taylor and N. Cristianini. Further results on the margin distribution. In *Proceedings of the Twelfth Annual Conference on Computational Learning Theory*, pages 278–285, 1999.

[31] J. Smith and S.-F. Chang. Automated image retrieval using color and texture. *IEEE Transaction on Pattern Analysis and Machine Intelligence*, November 1996.

[32] H. Tamura, S. Mori, and T. Yamawaki. Texture features corresponding to visual perception. *IEEE Transaction on Systems Man Cybernet (SMC)*, 1978.

[33] S. Tong and D. Koller. Support vector machine active learning with applications to text classification. *Proceedings of the 17th International Conference on Machine Learning*, pages 401–412, June 2000.

[34] V. Vapnik. *Estimation of Dependences Based on Empirical Data*. Springer Verlag, 1982.

[35] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.

[36] J. Wang, J. Li, and G. Wiederhold. Simplicity: Semantics-sensitive integrated matching for picture libraries. *ACM Multimedia Conference*, 2000.

[37] L. Wu, C. Faloutsos, K. Sycara, and T. R. Payne. Falcon: Feedback adaptive loop for content-based retrieval. *The* 26th *VLDB Conference*, September 2000.